> “Give me the first person row that has the name Dave.” This exhibits a distinctly different approach to retrieval; we’re not certain up front that we’ll get a result. It’s entirely possible the result set may be empty. Thus, returning `nil` in the case of finders that search for one row and an empty array for finders that search for many rows is the natural, nonexceptional response.

Often, though, you need to read in rows based on criteria other than their primary key value. Active Record provides additional methods enabling you to express more complex queries.

SQL and Active Record

To illustrate how Active Record works with SQL, pass a string to the `where` method call corresponding to a SQL `where` clause. For example, to return a list of all orders for Dave with a payment type of “po,” we could use this:

```
pos = Order.where("name = 'Dave' and pay_type = 'po'")
```

The result will be an `ActiveRecord::Relation` object containing all the matching rows, each neatly wrapped in an `Order` object.

That’s fine if our condition is predefined, but how do we handle it when the name of the customer is set externally (perhaps coming from a web form)? One way is to substitute the value of that variable into the condition string:

```
# get the name from the form
name = params[:name]
# DON'T DO THIS!!!
pos = Order.where("name = '#{name}' and pay_type = 'po'")
```

As the comment suggests, this isn’t a good idea. Why? It leaves the database wide open to something called a *SQL injection* attack, which the Ruby on Rails Guides[153] describe in more detail. For now, take it as a given that substituting a string from an external source into a SQL statement is effectively the same as publishing your entire database to the whole online world.